Rajshahi University Of Engineering and Technology

Department of Ceramic & Metallurgical Engineering

Assignment Report

Course Code:

Course Title:

Report Title:

“ Matrix Inversion using Recursion in C”

Submitted By

Taufiqe Emroze

ID:

Reg no:

Session:

Section:

Submitted To

Professor

Department of Computer Science and Engineering

Rajshahi University Of Engineering and Technology

Submission Date: 4th June 2026

1. Introduction:

This program finds the inverse of a square matrix of any dimension (n x n) using C. It takes the matrix size and its elements as input, then calculates and displays the determinant, the cofactor matrix, the adjoint (adjugate) matrix, and finally the inverse matrix.

Matrix inversion is a very common operation in mathematics, engineering, and computer science. It shows up in solving systems of linear equations, computer graphics, machine learning, and many other areas. This assignment helped us understand how to break down a large mathematical problem into smaller functions in C.

1.1 What is a Matrix Inverse?

For a square matrix A, the inverse is written as A^-1 and it satisfies the condition:

$$A \times A^{-1} = I \quad \text{(where I is the Identity Matrix)}$$

The inverse only exists if the determinant of A is not zero. If det(A) = 0, the matrix is called a singular matrix and cannot be inverted.

1.2 Formula Used

The formula for finding the inverse of a matrix is:

$$A^{-1} = Adj(A) / det(A)$$

Where Adj(A) is the adjoint matrix, which is the transpose of the cofactor matrix of A.

2. Algorithms

2.1 Algorithm: Determinant

The determinant is calculated using cofactor expansion (Laplace expansion) along the first row. For a matrix of size n, it recursively breaks the problem down into smaller (n-1) x (n-1) matrices.

Steps:

1. If n == 1, return arr[0][0].
2. If n == 2, return (arr[0][0]*arr[1][1]) - (arr[1][0]*arr[0][1]).
3. For each element in the first row (j = 0 to n-1):
4. Extract the mini matrix by removing row 0 and column j.
5. Multiply the element by its sign (+/-) and the determinant of the mini matrix.
6. Alternate the sign after each column.
7. Add up all the products to get the final determinant.

2.2 Algorithm: Cofactor Matrix

The cofactor matrix stores the signed determinant of the minor matrix for every position (i, j).

Steps:

8. For each element (i, j) in the matrix:
9. Extract the mini matrix by removing row i and column j.
10. Calculate the determinant of that mini matrix.
11. Assign the sign: +1 if (i+j) is even, -1 if (i+j) is odd.
12. cofactor[i][j] = sign × det(mini matrix).
13. Return the complete cofactor matrix.

2.3 Algorithm: Adjoint Matrix

The adjoint (or adjugate) matrix is simply the transpose of the cofactor matrix.

Steps:

14. For each position (i, j):
15. adjoint[i][j] = cofactor[j][i] (swap row and column).
16. Return the adjoint matrix.

2.4 Algorithm: Inverse Matrix

The inverse matrix is found by dividing every element of the adjoint matrix by the determinant.

Steps:

17. Check if det(A) != 0. If det is 0, inversion is not possible.
18. For each element (i, j) in the adjoint matrix:
19. inverse[i][j] = adjoint[i][j] / det(A).
20. Return the inverse matrix.

3. Flowcharts

The following flowcharts describe the logic of the three main functions: Determinant, Cofactor, and Transpose (Adjoint).

3.1 Flowchart: Determinant Function

START: determinant(n, arr)

↓

◇ **n == 1?** ◇

↓

Yes → Return arr[0][0]

↓

◇ **n == 2?** ◇

↓

Yes → Return (arr[0][0]*arr[1][1]) - (arr[1][0]*arr[0][1])

↓

Initialize: det = 0, sign = 1

↓

Loop: j = 0 to n-1 (for each column in first row)

↓

Call mini(n, arr, mrr, 0, j) to extract minor matrix

↓

det += sign * arr[0][j] * determinant(n-1, mrr)

↓

sign = sign * (-1)

↓

◇ **j < n?** ◇

↓

Yes → Continue loop | No → Exit loop

↓

RETURN det

3.2 Flowchart: Cofactor Function

START: cofactor(n, arr, crr)

↓

Loop: i = 0 to n-1

↓

Loop: j = 0 to n-1

↓

Call mini(n, arr, mrr, i, j) to extract minor matrix

↓

◇ (i + j) % 2 == 0? ◇

↓

Yes → sign = +1 | No → sign = -1

↓

crr[i][j] = sign * determinant(n-1, mrr)

↓

◇ All (i, j) done? ◇

↓

No → Continue inner loop | Yes → Exit

↓

END: cofactor matrix crr is complete

3.3 Flowchart: Transpose / Adjoint Function

START: adjoint(n, crr, drr)

↓

Loop: i = 0 to n-1

↓

Loop: j = 0 to n-1

↓

drr[i][j] = crr[j][i] (swap rows and columns)

↓

◇ All (i, j) done? ◇

↓

No → Continue loop | Yes → Exit

↓

END: adjoint matrix drr is complete

4. How to Run the Program

4.1 Requirements

- A C compiler: GCC (recommended) or any C99-compatible compiler
- A terminal or command prompt
- The source file: matrix_inverse.c

4.2 Compiling the Program

Open a terminal and navigate to the folder where the source file is saved. Run the following command:

```
gcc matrix_inverse.c -o matrix_inverse
```

This will produce an executable file named matrix_inverse (or matrix_inverse.exe on Windows).

4.3 Running the Program

```
./matrix_inverse
```

On Windows:

```
matrix_inverse.exe
```

4.4 Sample Input and Output

Enter the following when prompted:

```
Enter the dimension of the matrix: 3
Enter the elements of the matrix:
1 2 3
0 1 4
5 6 0
```

Expected output:

```
The main matrix:
1 2 3
0 1 4
5 6 0

The determinant: 1

The cofactor matrix:
-24  20  -5
 18 -15   4
  5  -4   1

The adjoint matrix:
-24  18   5
 20 -15  -4
 -5   4   1

The inverse matrix:
-24.00  18.00   5.00
 20.00 -15.00  -4.00
 -5.00   4.00   1.00
```

4.5 Edge Cases

- If n = 0 or negative: The program prints 'Invalid dimension!' and exits.
- If det = 0: The program prints 'Matrix Inversion Not Possible!' since the matrix is singular.
- If n = 1: The inverse is simply 1/arr[0][0], which is handled as a special case.

5. Program Analysis

5.1 Function Breakdown

The program is divided into clean, single-purpose functions. This makes the code modular and easy to understand.

Function	Parameters	Purpose
inputMatrix()	n, arr[n][n]	Takes matrix input from user
printMatrix()	n, arr[n][n]	Prints integer matrix to screen
printIMatrix()	n, arr[n][n]	Prints float inverse matrix
mini()	n, arr, mrr, row, col	Extracts minor matrix by removing a row and column
determinant()	n, arr[n][n]	Recursively calculates det(A)
cofactor()	n, arr, crr	Builds the cofactor matrix
adjoint()	n, crr, drr	Transposes cofactor to get adjoint
inverseM()	n, drr, irr, det	Divides adjoint by det to get inverse

5.2 Data Types

- The main matrix arr is stored as int[][].
- The cofactor matrix crr and adjoint drr are also int[][] since all intermediate steps use integer arithmetic.
- The inverse matrix irr uses float[][] because dividing by the determinant can produce decimal values.

5.3 Memory Usage

All matrices are declared as Variable Length Arrays (VLA) on the stack. This means the memory is automatically freed when the function returns. For very large matrices (n > ~50), the stack may overflow because VLA sizes are limited by the available stack space.

5.4 Recursion

The determinant() function calls itself recursively. Each call creates a new minor matrix of size (n-1) x (n-1). This continues until the base case (n == 1 or n == 2) is reached. The depth of recursion equals n-1 levels.

6. Time & Space Complexity Analysis

6.1 Determinant Function

The determinant function is the most expensive one because it is recursive and is also called repeatedly by the cofactor function.

- For each call, it calls itself n times.
- Each of those n calls itself (n-1) times, and so on.
- This creates a recursion tree with n! leaves (n factorial).

Component	Time Complexity	Space Complexity
determinant()	O(n!)	$O(n^2)$ stack depth
cofactor()	$O(n^3 \times (n-1)!)$	$O(n^2)$
adjoint()	$O(n^2)$	$O(n^2)$
inverseM()	$O(n^2)$	$O(n^2)$
OVERALL	**$O(n! \times n^2) \approx O(n!)$**	**$O(n^2)$**

Note: This implementation is not efficient for large matrices. For n > 10, the program will be very slow because of the O(n!) growth rate. More efficient methods like LU Decomposition ($O(n^3)$) exist but are more complex to implement.

6.2 Growth Rate Comparison

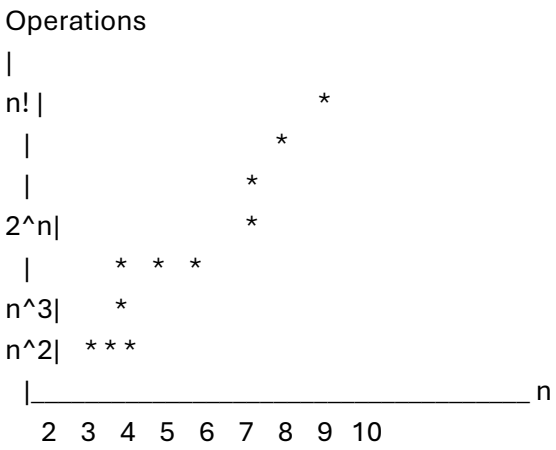
The table below compares the approximate number of operations for different n values:

n	n^2	n^3	2^n	n!	det() calls
2	4	8	4	2	2
3	9	27	8	6	9
4	16	64	16	24	64
5	25	125	32	120	325
6	36	216	64	720	1956
8	64	512	256	40320	109601
10	100	1000	1024	3628800	9864101

As the table shows, n! grows extremely fast. For n = 10, there are nearly 10 million determinant calls, which is why this recursive approach is only practical for small matrices (up to around n = 7 or 8).

6.3 Complexity Graph (Text Representation)

The graph below shows how each complexity class grows relative to n:



The n! line shoots up much faster than all the others, confirming that this algorithm is impractical for large inputs.

7. Limitations

- Only works for square matrices (n x n). Non-square matrices do not have an inverse.
- Uses integer arrays for the original matrix, so it does not support floating-point input values.
- The recursive determinant approach has O(n!) time complexity. For n > 8, the program becomes very slow.
- Uses Variable Length Arrays (VLA) which are stack-allocated. For large n, this can cause a stack overflow.
- No file I/O: input must be typed manually every time.

8. Future Works

There are many ways this program can be improved in the future:

21. Support float input: Change the matrix type from int to double so it can handle decimal inputs like 1.5, 0.3, etc.
22. Use LU Decomposition: Replace the recursive cofactor expansion with LU Decomposition. This brings time complexity down from O(n!) to $O(n^3)$, making it usable for larger matrices.
23. File Input/Output: Allow the user to read matrix data from a .txt file and save the output to another file, instead of typing it every time.
24. Error Handling for Overflow: Add a check for when n is too large and the program would likely crash or give wrong results.
25. Dynamic Memory Allocation: Replace VLAs with malloc() / free() to allocate matrices on the heap, which is safer for larger inputs.
26. GUI or Web Interface: Build a simple interface where users can enter a matrix visually and see the inverse displayed in a table format.
27. Batch Processing: Allow the user to input multiple matrices in one run and get the inverse of each one.
28. Support Non-square Matrices: Add the Moore-Penrose Pseudo-Inverse for rectangular matrices, which is widely used in data science and machine learning.

9. Conclusion

This program successfully implements matrix inversion in C using the standard mathematical method: computing the determinant, cofactor matrix, adjoint matrix, and then dividing by the determinant. It handles edge cases like singular matrices (det = 0), 1x1 matrices, and invalid dimensions.

The main takeaway is that breaking a big problem into small, focused functions makes the code clean and easier to debug. The program also shows clearly how recursion can be used to solve problems like determinant calculation, though the O(n!) complexity means it is not efficient for large matrices.

Working on this assignment helped practice important C concepts including: multi-dimensional arrays, Variable Length Arrays (VLA), recursion, and modular programming with functions.

End of Report